*electronics*

MDPI

*Article*

# RobotOBchain: Neighbor Observation for Byzantine Detection in Multi-Robot Systems

**Jie Luo [1,*], Yumeng Guo [1], Tiancheng Cao [2] and Wuyang Zhu [1]**

[1] Xiangjiang Laboratory, Changsha 410205, China; yumeng_guo@foxmail.com (Y.G.); zhuwuyang@alu.scu.edu.cn (W.Z.)
[2] School of Electrical and Electronic Engineering, Huazhong University of Science and Technology, Wuhan 430074, China; caotiancheng2025@163.com
* Correspondence: 2929@hutb.edu.cn

**Abstract**

Multi-robot systems are increasingly deployed in critical applications such as search and rescue, precision agriculture, and autonomous transportation. However, the presence of Byzantine robots—agents that intentionally transmit false or misleading information—can severely compromise mission success and system safety, highlighting the urgent need for robust fault-tolerant coordination mechanisms. To address the challenge of Byzantine faults in multi-robot systems, we propose a novel approach utilizing a blockchain-based framework, termed RobotOBchain (Robot Observation Blockchain). RobotOBchain permanently records each robot's own state information and its observed neighboring robots' states at every time step. By leveraging smart contracts encoded within the blockchain, our method automatically detects state inconsistencies or conflicts among recorded observations, enabling early identification of intentionally deceptive Byzantine robots. Experimental validation demonstrates that RobotOBchain achieves 100% consistent Byzantine identification across all robots, maintains estimation errors within 3% of ground-truth, and exhibits robust tolerance to up to 50% malicious agents. These results significantly surpass the performance of classical W-MSR algorithms, while eliminating the dependency on predefined fault bounds. The framework's demonstrated capabilities indicate strong potential for practical deployment in dynamic and safety-critical multi-robot applications.

**Keywords:** Byzantine detection; neighbor observation; blockchain smart contract; multi-robot consistency

check for updates

Academic Editor: Janos Botzheim

Received: 1 October 2025
Revised: 21 November 2025
Accepted: 4 December 2025
Published: 7 December 2025

**Citation:** Luo, J.; Guo, Y.; Cao, T.; Zhu, W. RobotOBchain: Neighbor Observation for Byzantine Detection in Multi-Robot Systems. *Electronics* **2025**, *14*, 4815. https://doi.org/10.3390/electronics14244815

**Copyright:** © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

## 1. Introduction

Multi-robot systems (MRSs) are rapidly transitioning from controlled laboratories to real-world safety-critical missions such as post-disaster search-and-rescue [1,2], precision agriculture [3,4], and autonomous last-mile delivery [5,6]. The fundamental promise of these systems—high redundancy, spatial parallelism, and efficient coordination—can only be realized if every member of the robot team behaves as expected. Unfortunately, the open nature of most robotic deployments exposes agents to malicious compromise. A robot that has been hijacked or reprogrammed can become Byzantine: it may intentionally broadcast false positions, forged sensor readings, or misleading status updates that rapidly propagate through the network, causing mission failure due to Byzantine faults, or even physical harm [7]. This makes Byzantine fault tolerance (BFT) research essential for building trust and robustness in MRSs. By enabling teams to securely coordinate despite malicious mem-

bers, BFT provides the critical foundation needed for the reliable, large-scale deployment of multi-robot systems in safety-critical missions.

Over the past decade, the robotics community has responded with a rich set of Byzantine fault-tolerant (BFT) coordination algorithms. The Weighted-Mean Subsequence Reduced (W-MSR) protocol [8] and its variants [9–11] provide Byzantine-resilient convergence by discarding the $F$ most extreme neighbor values. These methods are valued for their conceptual elegance and strong theoretical guarantees, ensuring the system's state converges to a safe value within the convex hull of the non-faulty agents' initial states. Nevertheless, a significant limitation arises in dynamic environments: the W-MSR family of algorithms is inherently more suitable for static networks with stable communication topologies, such as sensor networks, rather than for highly dynamic multi-robot systems where network connectivity changes frequently. This is because they require prior knowledge of the parameter $F$ (the maximum number of Byzantine agents) or strict connectivity maintenance to preserve it—an assumption that is rarely satisfied in real-world field deployments.

Blockchain-based approaches [12–14] have recently emerged as a complementary direction: by embedding consensus rules into smart contracts, swarms can achieve verifiable agreement without a central authority. This paradigm offers a powerful framework for achieving transparency, auditability, and decentralization, making the system's decision-making process tamper-evident. Current blockchain-based methods chiefly embed the W-MSR rule alongside blockchain functionalities, building on the work of Strobel et al. [7,15], and leverage the outlier-removal capability of W-MSR to neutralize interference from Byzantine robots. However, they are unable to pinpoint which specific robots are malicious or to reveal their actions. Moreover, the blockchains employed often rely on heavyweight consensus algorithms like Proof-of-Work, which are unsuitable for direct deployment on resource-constrained robots.

We therefore propose RobotOBchain, a lightweight blockchain framework designed to enhance security and accountability in multi-robot systems. The core idea is to transform real-time neighbor observations into immutable on-chain evidence, enabling smart contracts to autonomously detect Byzantine behavior and identify the specific malicious robots responsible. The framework operates through a coordinated on-chain and off-chain architecture. Each robot continuously records its own status and neighbor observations onto a Hashgraph-based distributed ledger, while ensuring communication security through an encrypted ROS protocol. These records form a tamper-proof audit trail of robot interactions. On the blockchain, smart contracts perform cross-validation by comparing each node's self-reported data with the majority observations of its neighbors. When an irreconcilable conflict is detected—such as a robot claiming one location while multiple neighbors report it elsewhere—the system automatically triggers a reputation mechanism, leading to the isolation of the Byzantine node.

Notably, this design requires no trusted hardware or central server, thus preserving the decentralized nature of robotic swarms. By leveraging a lightweight consensus mechanism, RobotOBchain achieves efficient and scalable collaborative verification, making it suitable for resource-constrained robotic platforms. The integration of on-chain conflict detection and off-chain data collection provides a balanced approach to resilience, enabling reliable identification of malicious actors without relying on predefined fault thresholds or static network assumptions. The main contributions of this work are summarized as follows:

1. RobotOBchain Framework: We propose a novel Byzantine detection framework that synergizes on-chain verification with off-chain neighbor observations. The system converts locally witnessed data into immutable evidence, enabling direct identification of malicious robots based on detectable behavioral inconsistencies while eliminating dependency on predefined fault thresholds.

2. Encrypted ROS Communication: We devise an asymmetric-cryptographic wrapper over ROS 2 topics that guarantees confidentiality, integrity, and source authentication of neighbor observations in real time. The mechanism eliminates single-point failures, resists plagiarism and tampering, and seamlessly feeds ciphertext observations into the Hashgraph consensus layer without altering the ROS publish/subscribe semantics.
3. Open-source Simulation Platform: We release a reproducible Webots-ROS-Hashgraph testbed and conduct systematic experiments under varying Byzantine densities. Without any a priori fault bound, the proposed pipeline consistently outperforms the W-MSR baseline in detection accuracy and convergence speed, validating its effectiveness in a high-fidelity robotic environment. The simulation suite is open-sourced at https://github.com/LuoJie996/RobotOBchain (accessed on 30 September 2025).

The remainder of this paper is organized as follows. Section 2 reviews the background and related work on Byzantine-resilient coordination algorithms and blockchain-based solutions for multi-robot systems. Section 3 details the proposed RobotOBchain methodology, including the on-chain/off-chain coordination protocol, smart contract logic, and corresponding security analysis. Section 4 presents the case study with experimental setup, evaluation metrics, and quantitative results. Section 5 analyzes the trade-offs between detection performance and system overhead, including communication cost and convergence time. Finally, Section 6 concludes with a summary of findings and discusses future research directions toward fully decentralized and incentive-compatible swarm operations.

## 2. Background and Related Work

In this section, we first discuss the multi-robot consensus problem and related methods, followed by an overview of fault detection in multi-robot systems (MRSs), and finally review recent advances in applying blockchain technology to achieve Byzantine resilience in MRSs.

### *2.1. Multi-Robot Consensus Achievement*

In multi-robot systems (MRSs), achieving consensus is one of the most fundamental requirements, underpinning a wide range of cooperative tasks such as collective perception [16,17], collective estimation [18], collective decision-making [19], and distributed flocking [20]. The best-known resilient consensus algorithms are the Linear Consensus Protocol (LCP) and its extension, the Weighted-Mean Subsequence Reduced (W-MSR) algorithm, which provides resilience against faulty or malicious robots [8]. The LCP operates through linear iterations, where each robot $i$ updates its state value as a weighted linear combination of its own value and those received from its neighbors [21]:

$$x_i(t+1) = w_{ii}(t)x_i(t) + \sum_{j\in\mathcal{N}_i(t)} w_{ij}(t)x_j(t), \tag{1}$$

where $t = 0, 1, 2, \ldots$ denotes discrete time steps, $w_{ij}(t)$ is the weight assigned by robot $i$ to neighbor $j$ at time $t$, and $\mathcal{N}_i(t)$ represents the set of neighbors of robot $i$. When uniform weights are used—specifically, $w_{ii}(t) = w_{ij}(t) = \frac{1}{|\mathcal{N}_i(t)|+1}$—this protocol reduces to average consensus, ensuring that all robots asymptotically converge to the average of their initial states [22]. To enhance robustness in adversarial environments, the W-MSR algorithm modifies the update rule by filtering out extreme values from neighboring robots before performing the state update. It assumes that at most $F$ of a robot's neighbors may be faulty. At each time step $t$, robot $i$ removes the $F$ largest and $F$ smallest values received from its neighbors and performs the update using only the remaining values:

$$x_i(t+1) = w_{ii}(t)x_i(t) + \sum_{j\in\mathcal{N}_i(t)\setminus\mathcal{Q}_i(t)} w_{ij}(t)x_j(t), \tag{2}$$

where $\mathcal{Q}_i(t) \subset \mathcal{N}_i(t)$ denotes the set of neighbors whose values are discarded by robot $i$ at time $t$. A key limitation of W-MSR is its need for prior knowledge of the upper bound $F$ on the number of faulty neighbors—a parameter that is typically unknown or difficult to estimate accurately in real-world deployments. In contrast, the method proposed in this paper does not require such prior knowledge, enabling more adaptive and scalable fault tolerance in dynamic and uncertain environments.

### *2.2. Fault Detection in MRSs*

Multi-robot systems, particularly swarm robotics, are often assumed to be inherently fault-tolerant due to redundancy. However, this assumption can fail when specific faults—such as sensor failures, software bugs, or adversarial behavior—propagate undetected [23]. A comprehensive survey on fault detection techniques in robotic swarms is provided by Khalastchi and Kaminka [24]. Early work focused on endogenous fault detection, where individual robots monitor their internal states to detect anomalies [25]. Over the past decade, research has increasingly focused on exogenous fault detection, which leverages inter-robot observations to identify deviations from expected behavior. These approaches typically compute residuals between observed and predicted behaviors and compare them against predefined thresholds to flag potential faults [26]. More recently, Wardega et al. [27] proposed an accusation-based framework in which robots generate local accusations upon observing misbehavior. These accusations are flooded throughout the network, and each robot independently constructs a blocklist using a graph-matching algorithm to identify consistently accused agents. This decentralized mechanism enables rapid isolation of faulty robots without relying on a central authority. Unlike W-MSR, the observation-based schemes reviewed above can detect robot failures without any prior knowledge. However, in the absence of network-wide connectivity, mutual fault detection that relies on propagating local observations faces two intertwined challenges: (1) incomplete dissemination yields inconsistent observation vectors across robots, so the Byzantine fault sets inferred by individual robots no longer coincide; (2) Byzantine agents situated on propagation paths can tamper with forwarded observations, further corrupting the detection results of healthy robots.

### *2.3. Byzantine Robot Detection via Blockchain*

Recent studies have explored the use of blockchain technology for detecting and mitigating Byzantine robots in multi-robot systems. By providing immutability, traceability, and decentralization, blockchains offer a promising infrastructure for secure coordination in open and untrusted environments. Strobel et al. [7] implemented the first proof-of-concept robot swarm that uses blockchain to manage Byzantine agents in a collective decision-making task. In their later work [18], they compared this approach with the W-MSR algorithm, demonstrating improved transparency and auditability in fault handling. Luo [28] extended this paradigm by introducing smart contracts to filter inconsistent data submitted by robots. The contract validates incoming measurements against consistency rules and excludes outliers, effectively isolating Byzantine agents exhibiting erratic behavior. Strobel et al. [15] recently proposed a token-based economy governed by blockchain. Smart contracts distribute tokens to robots based on their contributions during swarm operations. Robots engaging in disruptive behavior are penalized through token deduction, leading to eventual exclusion once their balance is depleted—a mechanism promoting accountability and long-term cooperation. Ferrer [29] utilized blockchain as an asynchronous message registry in follow-the-leader scenarios, preventing Byzantine robots from injecting or forwarding misleading commands. This ensures command integrity even under partial connectivity or delayed communication. However, real-world deployments face challenges

such as network partitions, which can delay consensus and create opportunities for attacks like the 51% attack. To address this, Keramat et al. [30] proposed replacing traditional blockchains with IOTA—a partition-tolerant distributed ledger based on Directed Acyclic Graph (DAG) architecture. Similarly, Salimpour et al. [31] applied IOTA in a vision-based Byzantine agent detection case study, highlighting its suitability for low-bandwidth and unreliable communication environments.

Existing research on leveraging blockchain technology to resist Byzantine attacks demonstrates common advantages, including avoiding single points of failure through a decentralized architecture, ensuring the auditability of all behavioral records by exploiting blockchain's immutability, and employing smart contracts to implement token-based reward and punishment mechanisms that incentivize compliant behavior. However, these methods still face limitations. For instance, their reliance on global consensus mechanisms leads to inefficiency during network partitions, and they fail to verify the reliability of raw observational data, making them susceptible to deception by forged data. Furthermore, the smart contract typically processes only high-level messages, overlooking rich local interaction information, while blockchain confirmation delays and high energy consumption constrain its use in real-time scenarios and on resource-constrained devices. To address these issues, RobotOBchain boosts consensus efficiency by leveraging an asynchronous Byzantine fault-tolerant (aBFT) consensus protocol and a roaming Ranger Robot relay mechanism. It ensures data authenticity in neighbor interactions using asymmetric encryption and adopts a method where neighbors conduct distributed, independent observations off-chain, while smart contracts perform globally consistent and fair detection on-chain.

## 3. Methodology

This section first overviews the off-chain/on-chain Byzantine detection framework, then details how neighbor observations are encrypted and gathered, how Hashgraph consensus enables reputation-based fault classification, and finally how the cleaned estimates are averaged online to achieve collective consensus.

### *3.1. Framework Overview*

We present the proposed framework as illustrated in Figure 1. To detail the approach framework, we take a case of a collective estimation scenario depicted in [7].

Here, we formulate the blockchain-based fault detection approach as two main parts, that is, off-chain stage and on-chain stage according to whether the action takes place off-chain or on-chain. The off-chain stage consists of two parallel steps. First, each robot independently senses the environment, converts raw sensor readings into an abstract "opinion" (e.g., tile color estimate), and packages this together with its own state into a local blockchain replica. Simultaneously, it observes every neighbor within range, assembles their reported positions and opinions into a neighbor-state set, and appends the same replica. Second, robots exchange these local replicas through encrypted ROS communication, propagating individual data to the swarm and forming a single, globally agreed history. Once consensus is reached, the smart contract retrieves the immutable data and performs online Byzantine detection. On-chain stage proceeds in four automated steps: (i) the locally deployed smart contract extracts the consensus-approved data from the robot's blockchain replica and reorganises it into a chronologically ordered set of data frames; (ii) for each frame, a conflict-check compares every robot's declared opinion against the neighbor observations stored in the same frame; (iii) support and opposition rates are computed from these cross-checks and used to update each robot's reputation via majority-based scoring; (iv) a final Byzantine verdict is reached by thresholding the accumulated reputation, yielding an immutable classification of faulty agents.

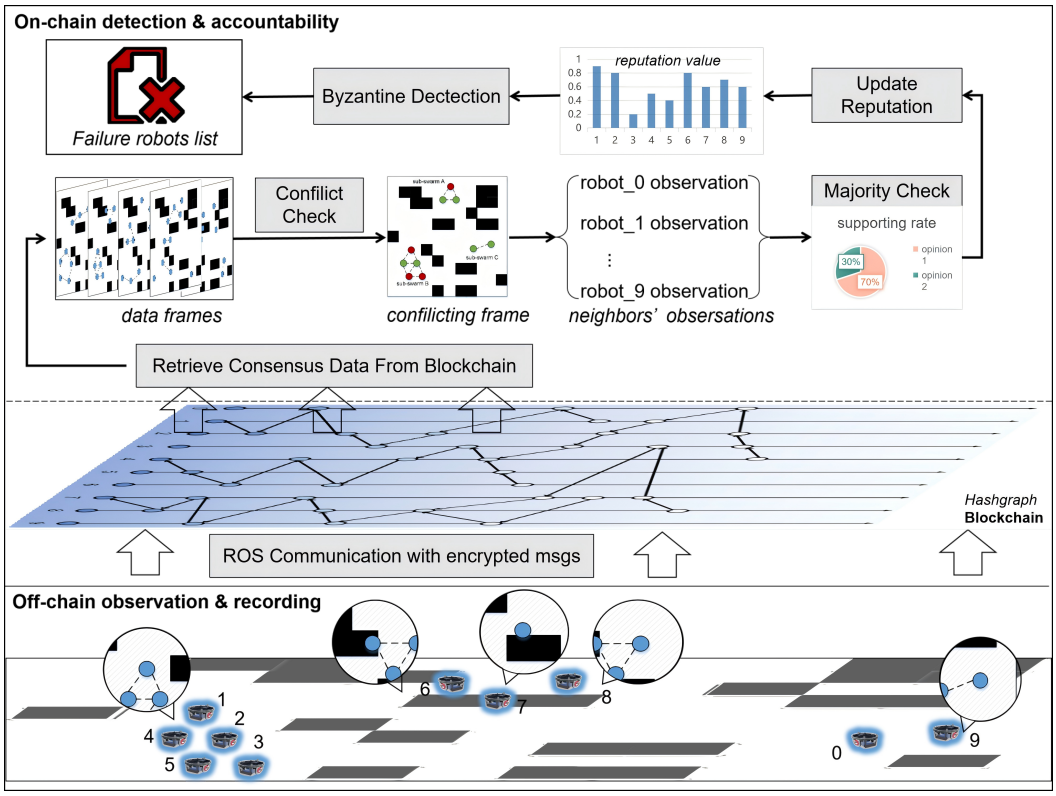



**Figure 1.** Approach Framework.

This approach utilizes a DAG-based blockchain technology, namely Hashgraph, to aggregate the observations of each robot. Thanks to its distributed nature, asynchronous consensus ability, and lightweight characteristics, the Hashgraph provides each robot with consistent historical spatiotemporal observation data, which serves as undeniable evidence for on-chain detection without consuming excessive computing resources compared to Proof-of-Work-based traditional blockchain technologies that are used in Bitcoin and Ethereum.

In addition, we introduce the Ranger Robot, a specialized agent designed for cross-region relay communication and accelerated consensus formation. It addresses consensus delays caused by limited communication range or network partitioning. Equipped with enhanced mobility and an extended communication radius, the Ranger Robot traverses between multiple sub-swarms, actively synchronizing Hashgraph data across regions to facilitate global consensus. This mechanism significantly improves consensus efficiency and robustness in sparsely distributed or dynamically partitioned environments. Detailed design, communication protocols, and experimental validation can be found in the paper [32]. The specifics of the Hashgraph consensus mechanism and the Ranger Robot implementation are not reiterated in the remainder of this paper.

The Hashgraph consensus mechanism employed in RobotOBchain provides inherent scalability advantages through its asynchronous Byzantine fault-tolerant (aBFT) properties. Unlike traditional blockchain consensus protocols that require synchronous communication rounds, Hashgraph's gossip-about-gossip protocol allows nodes to efficiently share information with a subset of neighbors, which then propagates through the network exponentially [33]. This design ensures that the communication complexity grows logarithmically with the number of nodes, making it suitable for medium-sized swarms of up to 100 robots.

To mitigate delays in consensus formation, our framework incorporates two key optimizations: (1) event batching, where multiple observations are aggregated into single

consensus events, reducing the frequency of consensus rounds; and (2) adaptive gossip intervals that adjust based on network density and message backlog. These mechanisms help maintain stable performance even as swarm size increases, though practical limitations exist for very large deployments as discussed in Section 6.

### *3.2. Off-Chain Observation*

We assume that each robot has a limited perception ability within a small range, allowing it to perceive the visible states (such as the position and the color of the tile at that position presented in our case study in Section 4) of neighboring robots. The neighboring robots observe each other's states and record this information into their respective blockchain replicas, ultimately forming immutable consensus data. The information recorded by the robot $i$ at each moment $t$ consists of a quintuple, i.e., $vote_i(t) = (i, t, p_i(t), o_i(t), \mathcal{N}_i(t))$, where $i$ represents the robot's identifier, $t$ represents the current timestamp, $p_i(t)$ represents the position where the robot locates at time $t$, $o_i(t)$ represents the opinion of robot $i$ at time $t$ (e.g., in our case study, it includes two elements, i.e., the color of the floor tile at current position $c_{p_i(t)}$ and the current estimate frequency of the black tile $x_i(t)$), and $\mathcal{N}_i(t)$ represents the observation list of robot $i$, which includes the states of all neighboring robots that robot $i$ sees at time $t$.

Each element in the observation list $\mathcal{N}_i(t)$ is a triad $(j, p_j^i(t), o_j^i(t))$, where $j$ is the identifier of a neighboring robot; the value $p_j^i(t)$ represents the neighboring robot $j$'s position at time $t$ from the perspective of robot $i$; and the value $o_j^i(t)$ represents the opinion that robot $j$ should hold at time $t$, from the perspective of robot $i$. For example, at the moment $t$, robot $i$ sees that robot $j$ is on a black tile. From the perspective of robot $i$, the opinion that robot $j$ should hold is "I am now on a black tile", i.e., $c_{p_i(t)} = 1$ (1 represents "black", while 0 represents "white"), regardless of what robot $j$ claims to be. At each time step $t$, the robot sends the quintuple $vote_i(t)$ to its blockchain interface which is implemented as an agent to connect the robot controller with the blockchain node.

If there are no other robots nearby, the blockchain interface will not save the robot's information to the local replica of the Hashgraph since the Hashgraph consensus algorithm requires each newly created event (namely block) to be linked to two-parent events through hash pointers, one of which is its previous latest event and the other is the latest event from another blockchain node. Instead, the $vote_i(t)$ data will be stored in a temporary list. Only when a neighboring robot appears, the data in the temporary list will be put into a Hashgraph blockchain replica. At the same time, the replica information will be synchronized with the neighboring robot. To prevent data from being intercepted by malicious nodes during transmission, we implemented an encrypted communication mechanism using ROS topics described in the following.

The use of ROS topic-based message publishing and subscription communication methods can meet the peer-to-peer and asynchronous connection characteristics of multi-robot networks. The latest ROS version has improved real-time performance and eliminated the single-point failure of the ROS master. To prevent the messages of normal robots from being plagiarized or tampered with by malicious nodes during transmission, this article applies asymmetric encryption methods to ROS communication as depicted in Figure 2.

Each robot generates a key pair consisting of a public and private (secret) key at the initial time and exchanges public keys with each other. These public and private keys are used for encrypting and decrypting messages in the subsequent process. Firstly, the sender encrypts a piece of plaintext to be submitted using the receiver's public key, resulting in ciphertext that can only be decrypted by the receiver using its private key. At the same time, the sender signs this plaintext with its private key and a hash function to create a digital signature, which also serves as a digital digest. Then, the sender packages the ciphertext

and digital signature into a serialized ROS message type and sends it to the corresponding topic "/sender_id/topic_name" associated with the sender's ID.

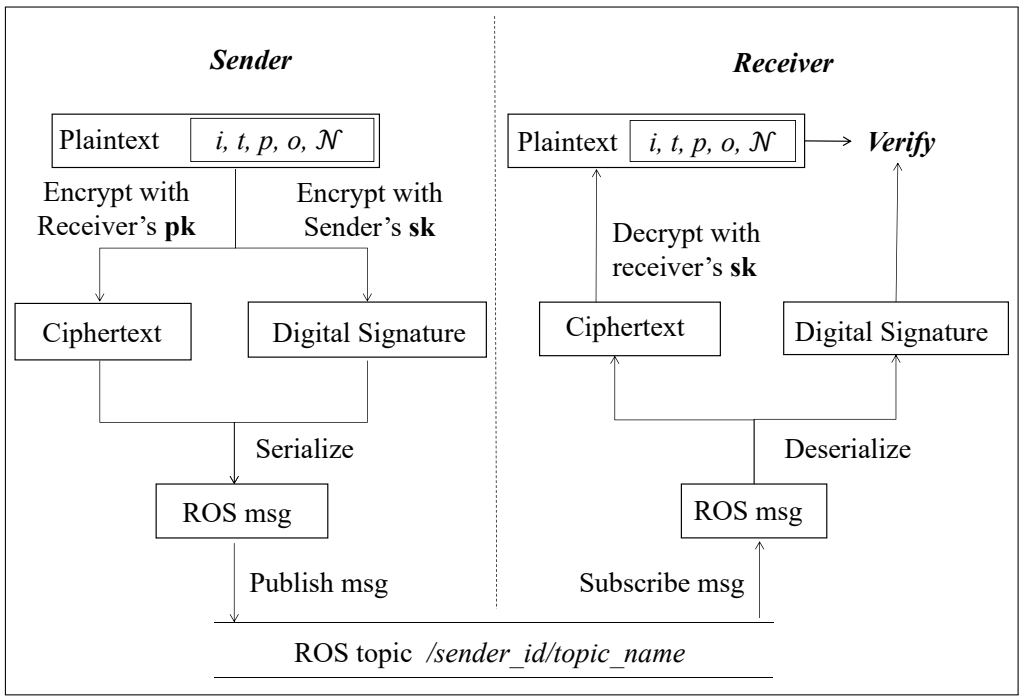



**Figure 2.** ROS Communication with Encryption.

The receiver constantly listens to this topic, and once a new message arrives, a callback function is invoked. This callback function subscribes to the messages of this topic, deserializes the ROS message into its original type, and obtains the corresponding ciphertext and digital signature. Subsequently, the receiver decrypts the ciphertext using its private key to obtain the plaintext. To verify the authenticity of the plaintext and that it has not been tampered with, the receiver passes the sender's public key, the decrypted plaintext, and the digital signature to a verification function. If the output of the verification function is "yes", that is, the relationship between these three parameters can be matched, then the plaintext is valid for later use.

The encrypted ROS communication layer is designed with scalability considerations. The asymmetric encryption overhead remains constant per message regardless of swarm size, as each robot only communicates with immediate neighbors within its communication radius. This local communication pattern ensures that the per-robot bandwidth requirements do not increase with overall swarm size, supporting scalable deployment in spatially distributed scenarios. However, the global consensus latency may still increase logarithmically with the number of nodes due to the gossip protocol's inherent properties [32].

### *3.3. On-Chain Detection*

Through the Hashgraph consensus algorithm, each robot's blockchain replica continuously aggregates data from neighboring robots encountered during the movement process and gradually reaches consensus on the data that has been added to the blockchain replica in earlier stages round by round, which is described in detail by the Hashgraph white paper [33].

When a new round of consensus is reached, this indicates that a batch of early data has been added to the consensus data. At this point, the smart contract will be triggered to detect the newly added consensus data. The first function of the smart contract is to retrieve

the data that has already reached consensus in the Hashgraph replica and rearrange the data in chronological order based on the timestamps. Each valid transaction is then inserted into the data frame by Algorithm 1 to construct the historical spatiotemporal observation data of the robot swarm.

**Algorithm 1** Consensus Data Retrieval

$df \leftarrow \emptyset$
**procedure** RETRIEVE_CONSENSUS_DATA($\mathcal{H}$)
  **for** each confirmed event $\mathcal{E}$ in $\mathcal{H}$ **do**
    **for** each transaction $tx$ in $\mathcal{E}$ **do**
      **if** $tx$ is not none **then**
        $i, t, p_i(t), o_i(t), \mathcal{N}_i(t) \leftarrow tx$
        **if** $tx$ not recorded in $df(t)$ **then**
          **insert** $(i, t, p_i(t), o_i(t), \mathcal{N}_i(t)$ **to** $df(t)$
        **end if**
      **end if**
    **end for**
  **end forreturn** $df$
**end procedure**

In Algorithm 1, an empty data frame list is defined at first, then a retrieve_consensus_data procedure is executed. Its input is the Hashgraph blockchain local replica. The algorithm checks for each transaction $tx$ in each confirmed event $\mathcal{E}$ in the Hashgraph replica $\mathcal{H}$, and inserts the quintuple $vote_i(t) = (i, t, p_i(t), o_i(t), \mathcal{N}_i(t))$ of each valid transaction to the data frame $df(t)$ whose time index is consistent with the timestamp in this quintuple.

A data frame is a two-dimensional table that records the states and observations of all robots at a certain time. Table 1 provides an example of a data frame.

**Table 1.** The Data Frame at Time $t$ ($df(t)$).

| Robot ID | Time | Position | Opinion | Observation |
|---|---|---|---|---|
| 1 | $t$ | $p_1$ | $o_1$ | $(2, p_2^1, o_2^1)$ $(3, p_3^1, o_3^1)$ |
| 2 | $t$ | $p_2$ | $o_2$ | $(1, p_1^2, o_1^2)$ |
| 3 | $t$ | $p_3$ | $o_3$ | $(1, p_1^3, o_1^3)$ |
| 4 | $t$ | $p_4$ | $o_4$ | $\emptyset$ |

The data frames with different time indexes together form a collection of historical spatiotemporal data about the state and observations of the entire robot swarm. The content of each data frame will be checked by the fault_detection function in chronological order according to the time index. The procedure is depicted in Algorithm 2.

Here, we make a conflict_check on the position and opinion of each robot recorded in each data frame, using the observed values of other neighboring robots recorded in the same data frame. First, we obtain the position $p_i(t)$ and opinion $o_i(t)$ that robot $i$ claims to be at time $t$ from $df(t)$. Then, in the same data frame $df(t)$, we can find if other robots have observed this robot at time $t$ by checking the observation list of each robot. For each neighboring robot $j$ that has observed robot $i$, we further check whether in its observation list $\mathcal{N}_j(t)$, the position $p_i^j(t)$ and state $o_i^j(t)$ of robot $i$ observed by robot $j$ are in conflict with the position $p_i(t)$ and state $o_i(t)$ claimed by robot $i$.

**Algorithm 2** Reputation-Based Fault Detection

**procedure** FAULT_DETECTION($df$)
  **for** each data frame $df(t)$ in $df$ **do**
    **for** each robot $i$ in $df(t)$ **do**
      get i's position and opinion, $(p_i(t), o_i(t)) \leftarrow df(t)$
      **for** each robot $j(\neq i)$ **do**
        **if** $i$ is in $j$'s observation list $\mathcal{N}_j(t)$ **then**
          get j's observation, $(p_i^j(t), o_i^j(t)) \leftarrow \mathcal{N}_j(t)$
          **if** $(o_i^j(t) == o_i(t)) \wedge (p_i^j(t) == p_i(t))$ **then**
            $sups_i++$
          **else**
            $opps_i++$
          **end if**
        **end if**
      **end for**
      **if** $sups_i > opps_i$ **then**
        $\mathcal{R}_i(t+1) = \mathcal{R}_i(t) + 1$
      **else if** $sups_i < opps_i$ **then**
        $\mathcal{R}_i(t+1) = \mathcal{R}_i(t) - 1$
      **else**
        $\mathcal{R}_i(t+1) = \mathcal{R}_i(t)$
      **end if**
    **end for**
  **end for**
  **for** each robot $i$ **do**
    **if** $\mathcal{R}_i < \gamma$ **then**
      **add** $i$ to $\mathcal{Q}$
    **end if**
  **end for**
  **return** $\mathcal{Q}$ ▷ The set of detected faulty robots
**end procedure**

Based on the checking result, we can calculate the number of supporters and opponents of the statement that "Robot $i$ holds the opinion $o_i(t)$ at position $p_i(t)$ at time $t$". Next, the majority_check on the robot $i$ is performed according to the principle of majority rule and determines whether to increase or reduce the robot's reputation. Finally, a list of all robots' current reputations will be returned. This reputation list will then be used for the failure_check procedure. The failure_check procedure is based on comparing the robot's reputation with a pre-set reputation threshold $\gamma$. If the reputation of robot $i$, i.e., $R_i$, is less than the threshold $\gamma$, the robot $i$ will be added to the detected faulty robot set and its data will not be used for further consensus tasks.

The majority-based scoring mechanism was selected based on its alignment with fundamental Byzantine fault tolerance principles and practical implementation considerations. Unlike probabilistic methods that introduce statistical uncertainties or machine learning approaches requiring extensive training, majority voting provides deterministic outcomes essential for safety-critical robotic systems. This approach leverages the inherent advantage of swarm redundancy, where the collective observations of honest robots naturally outweigh malicious reports through simple yet effective counting mechanisms.

The reputation threshold $\gamma$ was optimized through iterative experimentation to balance detection sensitivity and system stability. Our analysis demonstrates that the cumulative nature of reputation scoring provides inherent robustness to threshold variations, as consistent behavioral patterns over multiple observations naturally filter out transient

anomalies. This design ensures that the system maintains reliable performance across a practical range of parameter values without requiring precise tuning.

### *3.4. Consensus Achievement on Frequency Estimate*

The consensus estimate is computed after fault detection. For each data frame $df(t)$, we calculate the average estimate

$$x(t) = \sum_{i \in \mathcal{N}_{df(t)} \setminus \mathcal{Q}} x_i(t), \tag{3}$$

where $\mathcal{N}_{df(t)}$ is the set of robots whose information are recorded in $df(t)$, and $\mathcal{Q}$ is the set of detected faulty robots. Subsequently, every $x(t)$ is fed—in chronological order—to an online algorithm that iteratively refines the mean, variance, and standard error as follows:

$$\bar{x}(t+1) = \bar{x}(t) + \frac{x(t) - \bar{x}(t)}{n+1}, \tag{4}$$

$$M_2(t+1) = M_2(t) + \big(x(t) - \bar{x}(t)\big)\big(x(t) - \bar{x}(t+1)\big), \tag{5}$$

$$s^2(t+1) = \frac{M_2(t+1)}{n}, \tag{6}$$

$$\sigma_{\bar{x}} = \frac{s(t+1)}{\sqrt{n}}. \tag{7}$$

The pseudo-code is summarized in Algorithm 3. When the standard error $\sigma_{\bar{x}}$ drops below a predefined threshold $\tau$, the algorithm terminates and outputs the final consensus frequency estimate $x^* = \bar{x}$.

**Algorithm 3** Online Consensus Frequency Estimation (post fault-detection)

**Input:**
$\mathcal{Q}$ — set of detected Byzantine robots
$\{df(t)\}_{t=1}^{T}$ — chronological data frames after fault detection
$\tau$ — convergence threshold for standard error

**Output:**
$x^*$ — final consensus frequency estimate

$n \leftarrow 0, \quad \bar{x} \leftarrow 0, \quad M_2 \leftarrow 0$ ▷ initialize statistics
**for** $t = 1$ **to** $T$ **do**
  $\mathcal{N}_{df(t)} \leftarrow$ robots recorded in frame $df(t)$
  $x(t) \leftarrow \sum_{i \in \mathcal{N}_{df(t)} \setminus \mathcal{Q}} x_i(t)$ ▷ non-faulty average
  $n \leftarrow n + 1$
  $\bar{x}_{\text{old}} \leftarrow \bar{x}$
  $\bar{x} \leftarrow \bar{x} + \dfrac{x(t) - \bar{x}}{n}$ ▷ update mean
  $M_2 \leftarrow M_2 + \big(x(t) - \bar{x}_{\text{old}}\big)\big(x(t) - \bar{x}\big)$ ▷ update squared diff.
  $s \leftarrow \sqrt{M_2/n}, \quad \sigma_{\bar{x}} \leftarrow s/\sqrt{n}$ ▷ std. error
  **if** $\sigma_{\bar{x}} < \tau$ **then**
    **break** ▷ convergence achieved
  **end if**
**end for**
**return** $x^* = \bar{x}$

## 4. Case Study

In this section, we present a systematic case study on collective environmental feature estimation using a robot swarm. The objective is for all robots to collaboratively determine the global frequency of black tiles across the arena, despite the potential presence of

Byzantine agents that attempt to disrupt the estimation process. We evaluate and compare the fault tolerance performance of our proposed RobotOBchain framework against the classical W-MSR algorithm under varying numbers of faulty robots. All experiments are conducted using the e-puck2 robot model within the Webots simulation environment, ensuring high-fidelity dynamics and sensor modeling.

### *4.1. Environment Setup*

The experimental task requires robots to explore an unknown environment, locally sample a spatial feature, and collectively agree on its global frequency. As depicted on the left of Figure 3, the arena is a 2 m $\times$ 2 m square table tiled with 10 cm $\times$ 10 cm black and white squares. Tile colors are randomly assigned according to an i.i.d. Bernoulli process with a fixed expected black frequency of 0.40, serving as the ground-truth frequency for evaluation. A new tile configuration is generated for each experimental run to eliminate map memorization and ensure statistical independence across trials. The table surface is matte and non-reflective, surrounded by a 10 cm opaque wall to block external visual distractions and reduce IR sensor noise. An overhead motion-capture system provides centimetre-level ground-truth localization for data logging and offline analysis; this system is not accessible to the robots during operation. Byzantine robots are permitted to falsify their local color measurements before sharing, allowing them to inject biased data and skew the collective estimate toward 0 or 1. This setup realistically captures both sensing uncertainty and adversarial interference encountered in distributed estimation tasks.

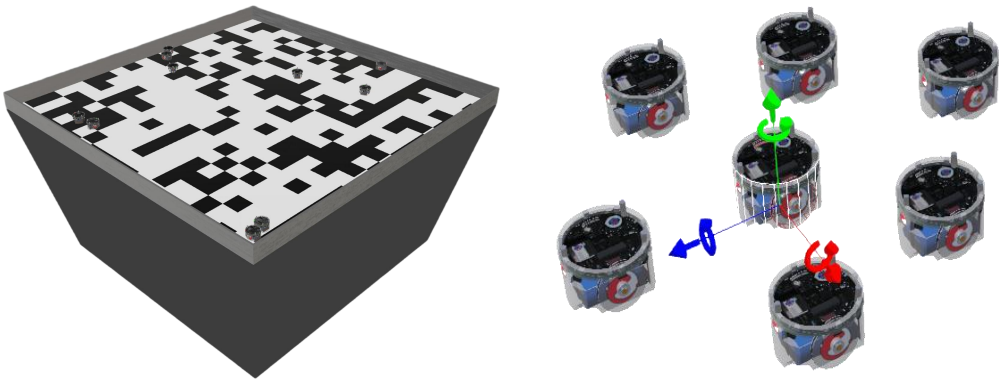


**Figure 3.** Simulation Scenario (**left**) and Robots (**right**).

### *4.2. Robot Setup*

Hardware Configuration. We deploy a team of 10 simulated e-puck2 miniature mobile robots as illustrated on the right of Figure 3.

Each unit has a cylindrical aluminium chassis (70 mm diameter, 45 mm height) and is equipped with

- A downward-facing VGA color camera for ground tile classification;
- Eight infra-red proximity and ambient-light sensors with a 6 cm detection range for obstacle avoidance and neighbor detection;
- A Bluetooth low-energy radio module enabling peer-to-peer communication within a 50 cm radius.

All sensing and communication modules are modeled with realistic noise profiles extracted from the official e-puck2 datasheet, ensuring that the simulation remains representative of real-world performance.

Control and Estimation Pipeline. Each robot executes a lightweight control loop consisting of two primary behaviors: (i) random walk, which periodically updates heading to ensure rapid spatial coverage, and (ii) bounce-away obstacle avoidance, triggered when

proximity sensors detect an obstacle closer than 5 cm. Concurrently, the vision module samples the center pixel of the downward camera at 10 Hz, classifies it as black or white using an adaptive threshold, and maintains a sliding 10 s histogram. The frequency of black samples within this buffer constitutes the robot's local frequency estimate $x_i(t)$. After an initial 10 s bootstrap phase, the robot packages its ID, timestamp, position, latest color sample, current estimate $x_i(t)$, and a list of detected neighbors into a ROS 2 message and transmits it to the blockchain interface at 1 Hz. This message format ensures that both first-hand sensor data and second-hand neighbor observations are available for on-chain consistency validation.

Byzantine Fault Behavior. Up to five robots may be compromised. A Byzantine agent mimics the motion and sampling routine of honest robots, but replaces the true color measurement with a deliberately biased value before computing $x_i(t)$. If multiple faulty robots are within communication range, they collude by exchanging their forged color readings and mutually updating their observation lists, thereby producing internally consistent yet globally misleading evidence. This coordinated attack model represents a worst-case scenario where adversaries leverage both sensor-level and data-level manipulation to degrade collective accuracy.

### *4.3. Performance Metrics*

We evaluate the quality of collective estimation through three complementary metrics, each capturing a distinct aspect of system behavior:

Estimate trajectory records the instantaneous black-tile frequency held by every robot throughout the mission. By plotting these curves together, we can visually inspect how quickly individual beliefs evolve, whether faulty agents manage to drag honest estimates away from the truth, and when the swarm settles on a common value.

Estimate error quantifies the final accuracy of the swarm. It is defined as the absolute difference between the ground-truth black frequency (0.4) and the consensus frequency produced by the algorithm after all transients have died out. To obtain a statistically reliable figure we average this error over ten independent runs, each with a freshly generated tile layout and random robot initialization.

Convergence time measures the speed of collective decision-making. We compute the earliest time at which the estimates of all non-Byzantine robots (or the entire swarm if Byzantine agents are present) remain within a $\pm 0.02$ band around their final mean value; this moment is reported as the convergence epoch. A shorter convergence time indicates that the system reaches a stable and coherent view of the environment more rapidly, a critical requirement for time-critical field applications.

### *4.4. Experimental Results*

We fix the W-MSR tolerance parameter at $F = 2$. As shown in Figure 4 (left), when the number of Byzantine robots $f \leq F$, W-MSR converges satisfactorily: honest robots settle around the ground-truth frequency (grey dashed line, 0.4), while faulty agents remain stuck at $x = 1.0$. Once $f > F$ (Figure 5 left), the algorithm loses resilience and the entire honest population is dragged toward the biased estimate ($x = 1.0$), yielding large steady-state error.

In contrast, the RobotOBchain approach with neighbor observation requires no prior bound on the number of faults. Byzantine records are first isolated by the smart contract; the remaining clean data are then averaged. Consequently, all robots converge to a value virtually identical to the true frequency ($\approx$0.4), demonstrating superior robustness regardless of $f$.

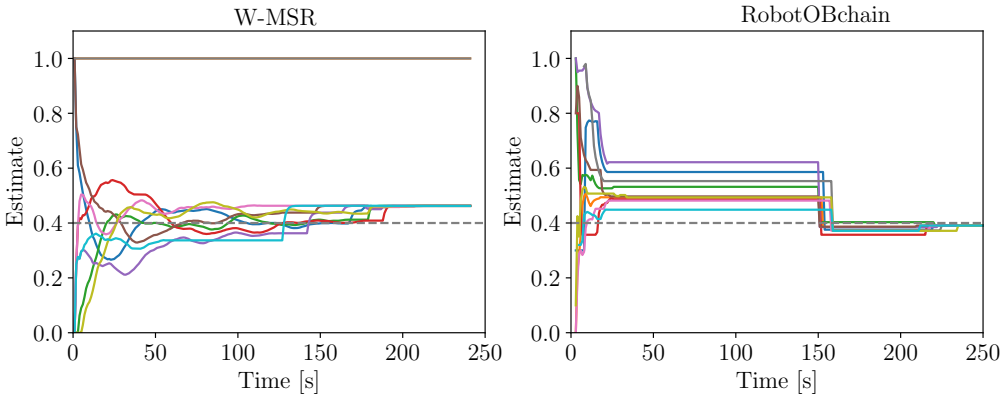



**Figure 4.** Frequency Estimate Convergence with $f = 2$ Byzantine Robots: W-MSR vs. RobotOBchain. The colored solid lines represent the frequency estimate trajectories of different robots, showing distinct convergence behaviors with tighter clustering under RobotOBchain indicating improved consensus among normal robots. The dashed line indicates the true frequency value.

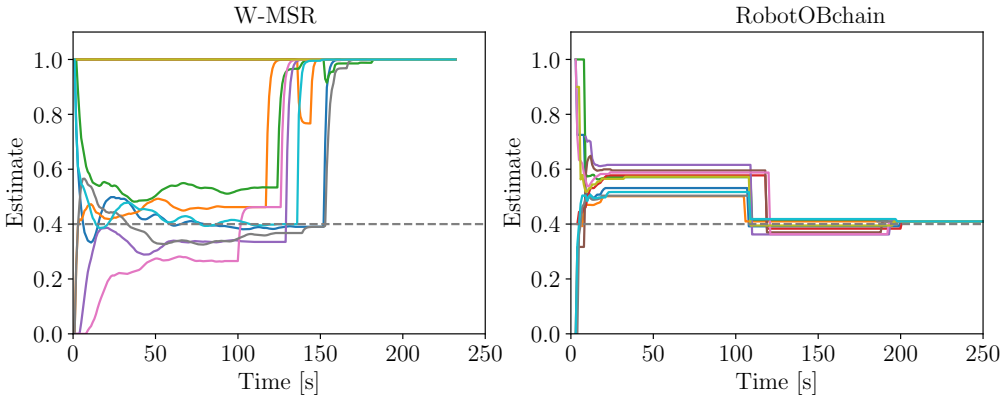



**Figure 5.** Frequency Estimate Convergence with $f = 3$ Byzantine Robots: W-MSR vs. RobotOBchain. he colored solid lines represent the frequency estimate trajectories of different robots, showing distinct convergence behaviors with tighter clustering under RobotOBchain indicating improved consensus among normal robots. The dashed line indicates the true frequency value.

Figures 6 and 7 summarize the quantitative results averaged over ten runs. Regarding estimate error (Figure 6), for W-MSR, it stays near zero while $f \leq 2$, but increases sharply once the actual fault count exceeds the design threshold $F = 2$. The RobotOBchain approach with neighbor observation maintains an error close to zero even when half of the team is malicious.

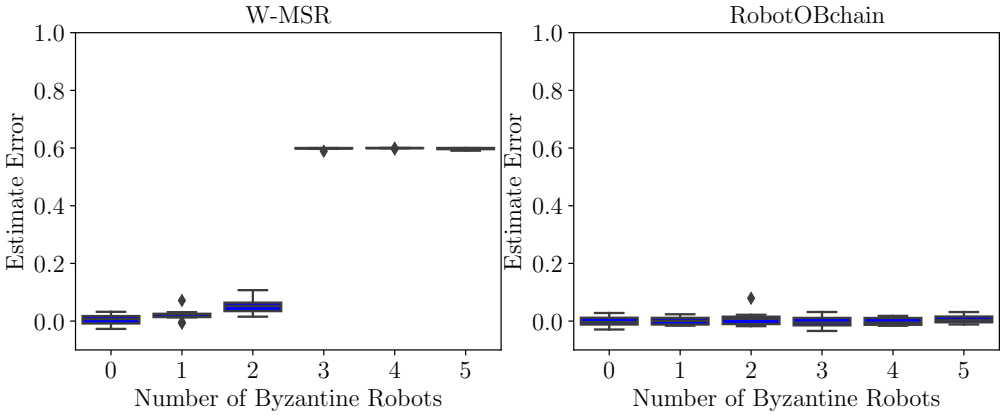



**Figure 6.** Average Estimate Error with Increasing Number of Byzantine Robots: W-MSR vs. RobotOBchain.

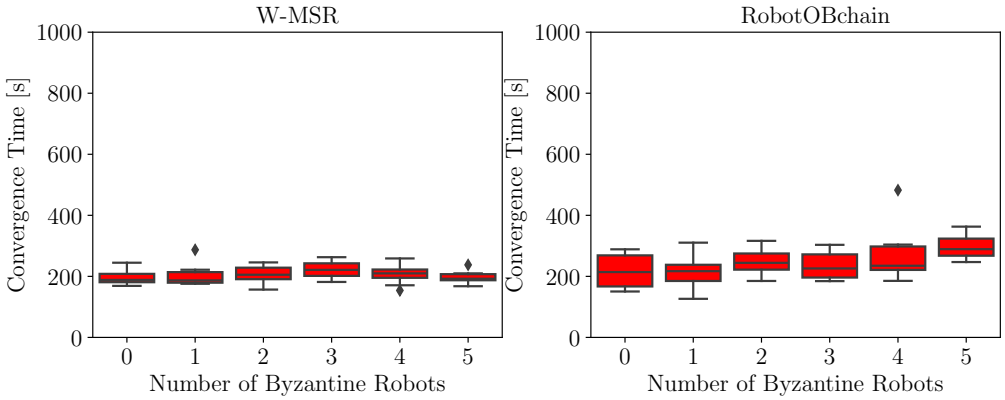



**Figure 7.** Convergence Time with Increasing Number of Byzantine Robots: W-MSR vs. RobotOBchain.

Regarding convergence time (Figure 7), the RobotOBchain approach with neighbor observation shows a mild increase with the number of Byzantine robots, whereas W-MSR remains almost flat. The difference is explained by the distinct termination criteria:

- W-MSR stops when the spread between the maximum and minimum estimate drops below a fixed tolerance; exact agreement is not required.
- The RobotOBchain approach with neighbor observation must (i) gather every estimate, (ii) reach global consensus via Hashgraph, and (iii) execute Byzantine detection before computing the final average—steps that naturally introduce a small time overhead.

Nevertheless, the additional delay is small: the blockchain solution achieves comparable speed to W-MSR while providing far better accuracy and unanimity.

Table 2 illustrates the consistency of the RobotOBchain approach with neighbor observation in a representative trial where robots 5 and 7 were intentionally configured as Byzantine. By the end of the run, every robot—honest or faulty—had identified the Byzantine set as $\{5,7\}$ and converged to the identical frequency estimate 0.3881, only 3% away from the ground-truth 0.4.

**Table 2.** Byzantine Robot Detection Results under the RobotOBchain.

| Method | Robot ID | Pre-Set as Byzantine? | Detected Byzantine Robot Set | Final Frequency Estimate |
|---|---|---|---|---|
| RobotOBchain | 0 | No | {5, 7} | 0.3881 |
| RobotOBchain | 1 | No | {5, 7} | 0.3881 |
| RobotOBchain | 2 | No | {5, 7} | 0.3881 |
| RobotOBchain | 3 | No | {5, 7} | 0.3881 |
| RobotOBchain | 4 | No | {5, 7} | 0.3881 |
| RobotOBchain | 5 | Yes | {5, 7} | 0.3881 |
| RobotOBchain | 6 | No | {5, 7} | 0.3881 |
| RobotOBchain | 7 | Yes | {5, 7} | 0.3881 |
| RobotOBchain | 8 | No | {5, 7} | 0.3881 |

For comparison, Table 3 shows an equivalent W-MSR trial where robots 6 and 8 were the actual adversaries. Because Byzantine units can tamper with their local diagnoses, the two faulty agents broadcast incompatible sets ($\{4,1\}$ and $\{9,3\}$), and even honest robots produce inconsistent lists (e.g., robot 4 outputs $\{2,6\}$) due to incomplete message propagation. Consequently, W-MSR fails to achieve a unique fault set or a single consensus value, highlighting its sensitivity to the parameter $F$.

Overall, the data confirm that W-MSR's fault tolerance frequency is tightly bounded by $P = F/N$; once real faults exceed $F$, adversaries dominate the estimate field. In contrast, the RobotOBchain approach with neighbor observation tolerates up to 50% Byzantine

robots without any prior tuning, while simultaneously delivering lower steady-state error, identical fault lists across the swarm, and only a marginal increase in convergence time.

**Table 3.** Byzantine Robot Detection Results under the W-MSR.

| Method | Robot ID | Pre-Set as Byzantine? | Detected Byzantine Robot Set | Final Frequency Estimate |
|---|---|---|---|---|
| W-MSR | 0 | No | {6, 8} | 0.4564 |
| W-MSR | 1 | No | {6, 8} | 0.4566 |
| W-MSR | 2 | No | {6, 8} | 0.4550 |
| W-MSR | 3 | No | {6, 8} | 0.4514 |
| W-MSR | 4 | No | {2, 6} | 0.4580 |
| W-MSR | 5 | No | {6, 8} | 0.4570 |
| W-MSR | 6 | Yes | {1, 4} | 1.0000 |
| W-MSR | 7 | No | {6, 8} | 0.4536 |
| W-MSR | 8 | Yes | {3, 9} | 1.0000 |
| W-MSR | 9 | No | {6, 8} | 0.4586 |

## 5. Discussion

This section discusses the trade-off between performance and overhead of the proposed neighbor-supervised Byzantine fault detection approach, including detection capability, communication and storage costs, and the resulting changes in consensus convergence time.

Performance and Overhead Analysis. The Byzantine fault detection method presented in this paper—grounded in off-chain neighbor supervision—bridges the gap left by conventional blockchain-based common-knowledge solutions that cannot prevent data tampering before it is written to the ledger. Traditional blockchain applications usually assume trustworthy data sources and only care about the immutability of on-chain data. In our scenario, however, the reliability of the data collection phase itself must also be guaranteed; hence we introduce a neighbor supervision mechanism. This mechanism improves failure detection performance, yet inevitably incurs additional overhead.

Detection Capability. Whenever a mismatch appears between a robot's self-reported state/view and the observations recorded by its neighbors, a Byzantine fault is guaranteed to exist (the faulty party may be either the observer neighbor or the uploading robot). In practice, as long as a Byzantine agent encounters an honest robot while roaming, its misbehavior will leave traceable evidence that will later be uploaded to the blockchain by the honest neighbor. Unless more than half of the neighbors' observations coincide with the Byzantine robot's self-report, the misbehavior will be detected; consequently, the method can theoretically tolerate up to 50% faulty robots, a figure consistent with the experimental results in the previous subsection. If a robot stops responding or sends ambiguous messages because of a byzantine fault, it can be detected directly at the consensus layer; the Hashgraph consensus adopted here can tolerate 33% such failures.

Communication and Storage Cost. To enable supervision, every robot must periodically upload both its own state and its observations of neighboring robots as auxiliary data, so that the authenticity of the consensus data used for common knowledge can be verified. This requirement introduces extra communication and storage costs. In our experiments, a robot that omits neighbor observations only needs to transmit the tuple $(i, t, p_i(t), o_i(t))$; if position and color are not checked, the message can be reduced to $(i, t, x_i(t))$. By contrast, uploading the full five-tuple $(i, t, p_i(t), o_i(t), \mathcal{N}_i(t))$ consumes significantly more bandwidth and storage. Quantitatively, transmitting estimates only costs 9 bytes per message. With the five-tuple, at least 21 bytes are required even when no neighbors are present; each additional neighbor adds another 12 bytes. Assuming an average of two neighbors per robot per time step, the message size grows to approximately 45 bytes—roughly four times the minimal overhead. Larger messages also increase the volume of data that the consensus

layer must process, lengthening convergence time, which explains the corresponding metric observed in the previous experiments. In short, we trade time for significantly higher fault tolerance—a classic compromise between computational resources and security.

Scalability Considerations. While the Hashgraph consensus provides better scalability than traditional blockchain approaches, practical limitations exist for very large swarms (e.g., >100 nodes). The gossip protocol's logarithmic scaling, while efficient, still introduces increased latency as network diameter grows. Our current implementation optimizes for typical swarm sizes in environmental monitoring and search-rescue scenarios, where 10–50 robots operate in moderate communication ranges. For massively large deployments, hierarchical consensus approaches or sharding techniques would be necessary to maintain real-time performance, representing an important direction for future work.

Impact of Communication Frequency and Network Latency. While the proposed framework demonstrates robustness against Byzantine faults, it is important to consider how communication dynamics affect detection accuracy and consensus latency. Higher reporting frequencies increase the volume of data being processed by the Hashgraph consensus layer, which may lead to longer convergence times, especially in bandwidth-constrained environments. Similarly, delayed message delivery—due to network congestion or intermittent connectivity—can temporarily desynchronize local observations, potentially deferring fault detection. However, since RobotOBchain already incorporates an adaptive reporting mechanism, future work will focus on further optimizing the rate adjustment logic and integrating delay-aware consensus protocols to better accommodate time-varying communication environments.

Robustness to Noisy or Imperfect Observations. The theoretical 50% fault tolerance assumes that neighbor observations are accurate and consistent. In practice, sensing noise or localization errors may introduce discrepancies between a robot's self-reported state and its neighbors' observations, potentially triggering false positives. To mitigate this, RobotOBchain employs a majority-based reputation mechanism that aggregates observations over time, reducing the impact of sporadic errors. Since detection is based on persistent inconsistencies rather than isolated mismatches, short-term observation noise is unlikely to destabilize fault classification. Future work will explore probabilistic observation models and adaptive reputation thresholds to further enhance detection reliability in noisy real-world environments.

From Technical Defense to Ethical Prevention. The RobotOBchain framework demonstrates that Byzantine resilience in multi-robot systems is technically achievable through decentralized neighbor observation and blockchain-based verification. However, technical solutions alone cannot prevent the creation of malicious code or harmful intentions. This limitation highlights the need for complementary ethical education in engineering curricula, focusing on risk assessment, dual-use technology awareness, and responsible innovation principles. By integrating technical safeguards with ethical training, we can develop MRSs that are both robust against attacks and trustworthy by design, addressing security challenges at both technological and human levels.

## 6. Conclusions

This paper presents RobotOBchain, a lightweight blockchain-enhanced framework that converts neighbor observations into immutable evidence to detect and isolate Byzantine robots without prior knowledge of fault counts or reliance on a central authority. By coupling off-chain encrypted ROS data collection with on-chain Hashgraph consensus and smart-contract-based reputation updates, the system achieves provable and unanimous identification of malicious agents even when up to 50% of the team is compromised.

Experimental results on a collective black-tile frequency estimation task demonstrate that RobotOBchain reduces the average estimate error to near zero, maintains convergence time comparable to the classic W-MSR baseline, and guarantees identical Byzantine lists across all robots—performance metrics that W-MSR cannot provide once the actual fault count exceeds its predefined threshold.

While the framework demonstrates significant potential for practical deployment, we acknowledge limitations in computational overhead and scalability to ultra-large swarms that warrant future optimization. Future work will extend the framework to dynamic topologies with intermittent connectivity, integrate adaptive reputation thresholds to further shorten convergence latency, and explore token-based incentive mechanisms that reward honest data sharing, paving the way toward fully decentralized, incentive-compatible and Byzantine-resilient robot swarms.

**Author Contributions:** Conceptualization, J.L. and Y.G.; methodology, J.L.; software, Y.G. and W.Z.; validation, Y.G., W.Z., and T.C.; formal analysis, J.L.; investigation, Y.G. and W.Z.; resources, J.L.; data curation, Y.G.; writing—original draft preparation, J.L. and Y.G.; writing—review and editing, T.C.; visualization, W.Z.; supervision, J.L.; project administration, J.L.; funding acquisition, J.L. All authors have read and agreed to the published version of the manuscript.

**Funding:** This work was supported by the General Project of Xiangjiang Laboratory under Grant 25XJ03014, Grant 25XJ03001, and Grant 25XJ03003, and by the Major Project of Xiangjiang Laboratory under Grant 24XJ01001.

**Data Availability Statement:** The code and data are available at https://github.com/LuoJie996/RobotOBchain (accessed on 30 September 2025).

**Conflicts of Interest:** The authors declare no conflicts of interest.

## References

1. Cao, X.; Li, M.; Tao, Y.; Lu, P. Hma-sar: Multi-agent search and rescue for unknown located dynamic targets in completely unknown environments. *IEEE Robot. Autom. Lett.* **2024**, *9*, 5567–5574. [CrossRef]
2. Kashyap, G.S.; Mahajan, D.; Phukan, O.C.; Kumar, A.; Brownlee, A.E.; Gao, J. From simulations to reality: Enhancing multi-robot exploration for urban search and rescue. *Int. J. Inf. Technol.* **2025**, *17*, 4261–4272. [CrossRef]
3. Ju, C.; Kim, J.; Seol, J.; Son, H.I. A review on multirobot systems in agriculture. *Comput. Electron. Agric.* **2022**, *202*, 107336. [CrossRef]
4. Yépez-Ponce, D.F.; Salcedo, J.V.; Rosero-Montalvo, P.D.; Sanchis, J. Mobile robotics in smart farming: Current trends and applications. *Front. Artif. Intell.* **2023**, *6*, 1213330. [CrossRef]
5. Wang, H.; Zhang, L.; Kong, Q.; Zhu, W.; Zheng, J.; Zhuang, L.; Xu, X. Motion planning in complex urban environments: An industrial application on autonomous last-mile delivery vehicles. *J. Field Robot.* **2022**, *39*, 1258–1285. [CrossRef]
6. Alverhed, E.; Hellgren, S.; Isaksson, H.; Olsson, L.; Palmqvist, H.; Flodén, J. Autonomous last-mile delivery robots: A literature review. *Eur. Transp. Res. Rev.* **2024**, *16*, 4. [CrossRef]
7. Strobel, V.; Castelló Ferrer, E.; Dorigo, M. Managing Byzantine Robots via Blockchain Technology in a Swarm Robotics Collective Decision Making Scenario. In Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems, Stockholm, Sweden, 10–15 July 2018; pp. 541–549.
8. LeBlanc, H.J.; Zhang, H.; Koutsoukos, X.; Sundaram, S. Resilient asymptotic consensus in robust networks. *IEEE J. Sel. Areas Commun.* **2013**, *31*, 766–781. [CrossRef]
9. Wang, Y.; Ishii, H. Resilient consensus through event-based communication. *IEEE Trans. Control Netw. Syst.* **2019**, *7*, 471–482. [CrossRef]
10. Sharifi, A.; Pourgholi, M. Resilient fixed-time bipartite consensus of multi-agent systems with nonlinear dynamics and directed graphs. *Proc. Inst. Mech. Eng. Part I J. Syst. Control Eng.* **2022**, *236*, 1755–1765. [CrossRef]
11. Guo, Y.; Pang, Y.; Lyons, J.; Lewis, M.; Sycara, K.; Liu, R. Trust-Aware Reflective Control for Fault-Resilient Dynamic Task Response in Human–Swarm Cooperation. *AI* **2024**, *5*, 446–464. [CrossRef]
12. Pacheco, A.; Zhao, H.; Strobel, V.; Roukny, T.; Dudek, G.; Reina, A.; Dorigo, M. Swarm Oracle: Trustless Blockchain Agreements through Robot Swarms. *arXiv* **2025**, arXiv:2509.15956. [CrossRef]
13. Yang, H. Swarm Contract: A Multi-Sovereign Agent Consensus Mechanism. *arXiv* **2024**, arXiv:2412.19256. [CrossRef]

14. Dorigo, M.; Pacheco, A.; Reina, A.; Strobel, V. Blockchain technology for mobile multi-robot systems. *Nat. Rev. Electr. Eng.* **2024**, *1*, 264–274. [CrossRef]
15. Strobel, V.; Pacheco, A.; Dorigo, M. Robot swarms neutralize harmful Byzantine robots using a blockchain-based token economy. *Sci. Robot.* **2023**, *8*, eabm4636. [CrossRef]
16. Valentini, G.; Brambilla, D.; Hamann, H.; Dorigo, M. Collective Perception of Environmental Features in a Robot Swarm. In Proceedings of the 10th International Conference on Swarm Intelligence (ANTS), Brussels, Belgium, 7–9 September 2016; Volume 9882, pp. 65–76.
17. Ebert, J.T.; Gauci, M.; Nagpal, R. Multi-Feature Collective Decision Making in Robot Swarms. In Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems, Stockholm, Sweden, 10–15 July 2018; Volume 9, pp. 1711–1719.
18. Strobel, V.; Castelló Ferrer, E.; Dorigo, M. Blockchain Technology Secures Robot Swarms: A Comparison of Consensus Protocols and Their Resilience to Byzantine Robots. *Front. Robot. AI* **2020**, *7*, 54. [CrossRef]
19. Reina, A.; Dorigo, M.; Trianni, V. Collective decision making in distributed systems inspired by honeybees behaviour. In Proceedings of the 13th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS 2014), Paris, France, 5–9 May 2014; Volume 2, pp. 1421–1422.
20. Ferrante, E.; Turgut, A.E.; Huepe, C.; Stranieri, A.; Pinciroli, C.; Dorigo, M. Self-organized flocking with a mobile robot swarm: A novel motion control method. *Adapt. Behav.* **2012**, *20*, 460–477. [CrossRef]
21. Xiao, L.; Boyd, S. Fast Linear Iterations for Distributed Averaging. In Proceedings of the 42nd IEEE International Conference on Decision and Control, Maui, HI, USA, 9–12 December 2003; pp. 4997–5002.
22. Xiao, L.; Boyd, S.; Lall, S. A Scheme for Robust Distributed Sensor Fusion Based on Average Consensus. In Proceedings of the Fourth International Symposium on Information Processing in Sensor Networks, Los Angeles, CA, USA, 25–27 April 2005; pp. 63–70.
23. Bjerknes, J.D.; Winfield, A.F.T. On Fault Tolerance and Scalability of Swarm Robotic Systems. In *Distributed Autonomous Robotic Systems*; Springer: Berlin/Heidelberg, Germany, 2013; Volume 83, pp. 431–444.
24. Khalastchi, E.; Kalech, M. Fault Detection and Diagnosis in Multi-Robot Systems: A Survey. *Sensors* **2019**, *19*, 4019. [CrossRef]
25. Zhang, X.; Zhang, Q. Distributed fault diagnosis in a class of interconnected nonlinear uncertain systems. *Int. J. Control* **2012**, *85*, 1644–1662. [CrossRef]
26. Kladis, G.P.; Tsourveloudis, N.C. Distributed Fault Detection For UAV Formation Missions. In Proceedings of the International Conference on Unmanned Aircraft Systems, Athens, Greece, 1–4 September 2020; pp. 1075–1084.
27. Wardega, K.; von Hippel, M.; Tron, R.; Nita-Rotaru, C.; Li, W. Byzantine Resilience at Swarm Scale: A Decentralized Blocklist Protocol from Inter-Robot Accusations. In Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems, London, UK, 29 May–2 June 2023; pp. 1430–1438.
28. Luo, J.; Ding, B.; Xu, J. Filtering Inconsistent Failure in Robot Collective Decision with Blockchain. In Proceedings of the IEEE/ASME International Conference on Advanced Intelligent Mechatronics, Hong Kong, China, 8–12 July 2019; pp. 577–582.
29. Ferrer, E.C.; Jimenez, E.; Lopez-Presa, J.L.; Martin-Rueda, J. Following Leaders in Byzantine Multirobot Systems by Using Blockchain Technology. *IEEE Trans. Robot.* **2021**, *38*, 1101–1117. [CrossRef]
30. Keramat, F.; Queralta, J.P.; Westerlund, T. Partition-Tolerant and Byzantine-Tolerant Decision-Making for Distributed Robotic Systems with IOTA and ROS 2. *arXiv* **2022**, arXiv:2208.13467.
31. Salimpour, S.; Keramat, F.; Queralta, J.P.; Westerlund, T. Decentralized Vision-Based Byzantine Agent Detection in Multi-Robot Systems with IOTA Smart Contracts. *arXiv* **2022**, arXiv:2210.03441.
32. Luo, J.; Shu, X.; Zhai, Y.; Fu, X.; Ding, B.; Xu, J. A Fast and Robust Solution for Common Knowledge Formation in Decentralized Swarm Robots. *J. Intell. Robot. Syst.* **2022**, *106*, 68. [CrossRef]
33. Baird, L. *The Swirlds Hashgraph Consensus Algorithm: Fair, Fast, Byzantine Fault Tolerance*; Swirlds Tech Report Swirlds-TR-2016-01; Swirlds, Inc.: Dallas, TX, USA, 2016.


**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.